

NexusDashboard.Connect.ps1

PowerShell script - reads the switches managed by Cisco Nexus Dashboard (Fabric Controller) over its REST API, logging in through the TACACS login domain, and adds them to SolarWinds as nodes with their pollers and interfaces, the way DNA.DiscoverNodesAndInterfaces.ps1 does it for Cisco DNA Center. Every run is logged.

736 lines · Scripts/NexusDashboard.Connect.ps1

```
1 #Requires -Version 7
2
3 <#
4 .SYNOPSIS
5     SolarWinds SWIS PowerShell script add the switches managed by Cisco Nexus Dashboard as nodes and
6     discover their interfaces
7
8 .DESCRIPTION
9     This script combines the Cisco Nexus Dashboard (ND) REST API with two
10    building blocks from the SolarWinds SDK samples, the same way
11    DNA.DiscoverNodesAndInterfaces.ps1 does it for Cisco DNA Center:
12
13    o Nexus Dashboard REST API
14      - authenticates (POST /login) against the login domain given in
15        -NXDomain and keeps the returned JWT for follow-up calls;
16      - refreshes the token (POST /refresh) before it expires - ND tokens
17        are valid for 20 minutes by default;
18      - reads the switch inventory managed by the Fabric Controller (NDFC)
19        service;
20      - logs out (POST /logout) as soon as the inventory is read, so the
21        token is revoked rather than left valid while the nodes are added.
22
23    o CRUD.AddNode.ps1
24      - adds a node <component> using CRUD operations (New-SwisObject) and
25        registers the standard set of pollers for it (Status, Response
26        Time, Details, Uptime).
27
28    o NPM.DiscoverAndAddInterfacesOnNode.ps1
29      - uses Orion.NPM.Interfaces.DiscoverInterfacesOnNode and
30        Orion.NPM.Interfaces.AddInterfacesOnNode (SWISv3 verbs, NPM only)
31        to discover and add the interfaces of a node.
32
33    The result is a single script that walks the switches Nexus Dashboard
34    reports, adds every one of them that does not already exist in
35    SolarWinds by its management IP address - with the switch name as the
36    node caption - and then discovers and adds their interfaces for
37    monitoring. -InterfaceFilter decides which interfaces are kept.
38
39    Every run records each switch returned, each component that was added,
40    already existed, was skipped or failed, every interface that was added,
41    and every poller that was registered, followed by a summary line.
42
43    Please update the parameter defaults below to match your environment
44    before running.
45
46 .PARAMETER NXServer
47     Host name or IP address of the Nexus Dashboard (the cluster's management
48     address). The API is reached on the default HTTPS port (443).
49
50 .PARAMETER NXCredentialPath
51     Path to a credential saved with Export-Clixml, for unattended runs
52     (scheduled task). Pass an empty string to be prompted instead. Create
53     the file once, as the account the task runs under:
54     Get-Credential | Export-Clixml -Path D:
55     \SolarWindsScripts\PowerShell\Credentials\NexusDashboard_Credential.xml
56
57 .PARAMETER NXDomain
```

```

56 The ND login domain the user authenticates against. "TACACS" is the
57 TACACS+ login domain; it must match the domain name configured in ND
58 (Admin > Authentication > Login Domains) exactly, case included. Use
59 "DefaultAuth" for a user from the local ND user database.
60
61 .PARAMETER TrustAllCertificates
62     Accept the self-signed certificate of the Nexus Dashboard. Meant for
63     testing; import the certificate into the trust store for production.
64
65 .PARAMETER TokenRefreshMinutes
66     Age, in minutes, after which the ND token is renewed before the next
67     call. Must stay below the token lifetime configured in ND (20 minutes by
68     default).
69
70 .PARAMETER SwisHost
71     Address of the SolarWinds server (SWIS).
72
73 .PARAMETER SwisCredentialPath
74     Path to a PSCredential exported with Export-Clixml for a SolarWinds user
75     that can add nodes.
76
77 .PARAMETER EngineID
78     SolarWinds polling engine the new nodes are assigned to.
79
80 .PARAMETER SNMPVersion
81     SNMP version used to poll the new nodes.
82
83 .PARAMETER SNMPCommunity
84     SNMP community configured on the switches. Without it the Details/Uptime
85     pollers and the interface discovery have nothing to poll.
86
87 .PARAMETER InterfaceFilter
88     Regular expression matched against the interface caption. Only the
89     interfaces that match it and are operationally up (ifOperStatus 1) are
90     added. The default keeps the NX-OS physical ports and port-channels
91     (Ethernet1/1, Eth1/1, port-channel10, Po10); everything else - mgmt0,
92     Vlan, loopback, nve and the like - is left out. The match is not case
93     sensitive.
94
95 .PARAMETER LogPath
96     The log file this script writes to. It is bounded by rotation rather
97     than by age, the same way DNA.DiscoverNodesAndInterfaces.ps1 bounds its
98     own log.
99
100 .PARAMETER LogMaxBytes
101     The size at which the log is rotated. Rotation keeps LogKeep older
102     generations beside it, so the space the logging can occupy is bounded at
103     roughly LogMaxBytes times LogKeep plus one.
104
105 .PARAMETER LogKeep
106     How many rotated logs to keep. Zero keeps none: the log starts over
107     instead.
108
109 .EXAMPLE
110     .\NexusDashboard.Connect.ps1
111
112     Runs with the defaults set in the param block.
113
114 .EXAMPLE
115     .\NexusDashboard.Connect.ps1 -NXServer nd.example.local -SwisHost orion.example.local `
116         -SNMPCommunity "mycommunity" -TrustAllCertificates
117
118 .NOTES
119     Requires PowerShell 7+ for the -SkipCertificateCheck switch on
120     Invoke-RestMethod, and the SwisPowerShell module for Connect-Swis.

```

```

121#>
122
123param(
124    # --- Cisco Nexus Dashboard connection ---
125    [string]$NXServer = "1.1.1.1",
126    [string]$NXCredentialPath = "D:\SolarWindsScripts\PowerShell\Credentials\FMC_Credential.xml",
127    [string]$NXDomain = "TACACS",
128
129    [switch]$TrustAllCertificates,
130
131    [ValidateRange(1, 19)]
132    [int]$TokenRefreshMinutes = 15,
133
134    # --- SolarWinds connection ---
135    [string]$SwisHost = "5.5.5.5",
136    [string]$SwisCredentialPath = "D:\SolarWindsScripts\PowerShell\Credentials\SolarWindsCredentials.xml",
137
138    # Shared settings applied to every switch retrieved from Nexus Dashboard.
139    [int]$EngineID = 2,
140    [int]$SNMPVersion = 2,
141    [string]$SNMPCommunity = '',
142    [string]$InterfaceFilter = '^(Ethernet|Eth\d|port-channel|Po\d)',
143
144    # --- Log ---
145    [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\nexusdashboard-discover-
nodes.log'),
146
147    [ValidateRange(4KB, 100MB)]
148    [int] $LogMaxBytes = 1MB,
149
150    [ValidateRange(0, 20)]
151    [int] $LogKeep = 3
152)
153
154# --- Logging ---
155# Taken from DNA.DiscoverNodesAndInterfaces.ps1: one log file bounded by
156# size-based rotation, the same level set and line format, and the same lazy
157# creation of the log directory on first write. The file calls carry
158# -ErrorAction Stop of their own, since this script does not set
159# $ErrorActionPreference = 'Stop'; without it a failed write escapes the
160# catch as a raw error instead of the intended warning.
161
162function Invoke-LogRotation {
163    # Renames the log out of the way once it reaches -LogMaxBytes, keeping
164    # -LogKeep older generations: .log -> .log.1, .log.1 -> .log.2, and so on,
165    # with the oldest dropped. Run on a schedule the log would otherwise grow
166    # without bound.
167    param(
168        [Parameter(Mandatory)][string] $Path
169    )
170
171    if (-not (Test-Path -LiteralPath $Path)) { return }
172    if ((Get-Item -LiteralPath $Path).Length -lt $script:LogMaxBytes) { return }
173
174    # Nothing to keep: the log simply starts over.
175    if ($script:LogKeep -lt 1) {
176        Remove-Item -LiteralPath $Path -Force -WhatIf:$false -Confirm:$false -ErrorAction Stop
177        return
178    }
179
180    # Oldest generation falls off the end.
181    $oldest = "$Path.($script:LogKeep)"
182    if (Test-Path -LiteralPath $oldest) {
183        Remove-Item -LiteralPath $oldest -Force -WhatIf:$false -Confirm:$false -ErrorAction Stop
184    }

```

```

185
186 # Shift the rest down, highest first so nothing is overwritten on the way.
187 for ($i = $script:LogKeep - 1; $i -ge 1; $i--) {
188     $from = "$Path.$i"
189     if (Test-Path -LiteralPath $from) {
190         Move-Item -LiteralPath $from -Destination "$Path.($i + 1)" -Force -WhatIf:$false -Confirm:$false
191     }
192 }
193
194 Move-Item -LiteralPath $Path -Destination "$Path.1" -Force -WhatIf:$false -Confirm:$false -ErrorAction
195 Stop
196
197 function Write-Log {
198     <#
199     .SYNOPSIS
200         Record one line in the run log, and on the console unless suppressed
201     .PARAMETER NoConsole
202         Write to the log file only. Used for the per-item detail lines - every
203         switch, component, interface and poller - which belong in the record
204         but would bury the console summary.
205     #>
206     param(
207         [Parameter(Mandatory)]
208         [AllowEmptyString()]
209         [string] $Message,
210
211         [ValidateSet('INFO', 'WARN', 'ERROR')]
212         [string] $Level = 'INFO',
213
214         [switch] $NoConsole
215     )
216
217     $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
218     if (-not $NoConsole) {
219         Write-Information -MessageData $line -InformationAction Continue
220     }
221
222     # -WhatIf:$false so that the log keeps being written even if a caller
223     # passes -WhatIf through; the log is a record, never one of the changes
224     # being previewed.
225     try {
226         $directory = Split-Path -Path $script:LogPath -Parent
227         if ($directory -and -not (Test-Path -LiteralPath $directory)) {
228             New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false -ErrorAction
229             Stop | Out-Null
230         }
231         Invoke-LogRotation -Path $script:LogPath
232         Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false -
233         ErrorAction Stop
234     } catch {
235         Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
236     }
237 }
238
239 function Get-NexusDashboardErrorMessage {
240     # Turns a failed Invoke-RestMethod into one readable line: the HTTP status
241     # plus the error text ND puts in the body, when there is one.
242     param(
243         [Parameter(Mandatory = $true)] [System.Management.Automation.ErrorRecord]$ErrorRecord
244     )
245
246     $status = $null

```

```

246 if ($ErrorRecord.Exception.Response) {
247     $status = [int]$ErrorRecord.Exception.Response.StatusCode
248 }
249
250 $detail = $ErrorRecord.ErrorDetails.Message
251 if (-not $detail) { $detail = $ErrorRecord.Exception.Message }
252
253 if ($status) { "HTTP ${status}: $detail" } else { $detail }
254}
255
256function Connect-NexusDashboard {
257    param(
258        [Parameter(Mandatory = $true)] [string]$Server,
259        [Parameter(Mandatory = $true)] [string]$Username,
260        [Parameter(Mandatory = $true)] [System.Security.SecureString]$Password,
261        # Login domain. "DefaultAuth" is the local user database; for a remote
262        # (RADIUS/TACACS/LDAP) user pass the login domain name configured in ND.
263        [string]$Domain = "DefaultAuth",
264        [switch]$TrustAllCertificates,
265        # Renew the token once it is older than this, ahead of the 20 minute
266        # default expiry.
267        [int]$TokenRefreshMinutes = 15
268    )
269
270    # PtrToStringBSTR (rather than PtrToStringAuto) decodes the BSTR as UTF-16 on
271    # every platform, and the buffer is zeroed again right after it is read.
272    $passwordPtr = [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($Password)
273    try {
274        $plainPassword = [System.Runtime.InteropServices.Marshal]::PtrToStringBSTR($passwordPtr)
275    }
276    finally {
277        [System.Runtime.InteropServices.Marshal]::ZeroFreeBSTR($passwordPtr)
278    }
279
280    $baseUri = "https://$Server"
281
282    $body = @{
283        userName    = $Username
284        userPasswd  = $plainPassword
285        domain      = $Domain
286    } | ConvertTo-Json
287
288    $params = @{
289        Uri          = "$baseUri/login"
290        Method       = "Post"
291        Body         = $body
292        ContentType  = "application/json"
293        SessionVariable = "webSession"
294    }
295    if ($TrustAllCertificates) {
296        $params["SkipCertificateCheck"] = $true
297    }
298
299    try {
300        $response = Invoke-RestMethod @params
301    }
302    catch {
303        throw "Login to $Server failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
304    }
305
306    # The token is returned as "jwttoken" (older releases also mirror it in
307    # "token") and set as the AuthCookie cookie; any of the three will do.
308    $token = $response.jwttoken
309    if (-not $token) { $token = $response.token }
310    if (-not $token) {

```

```

311     $cookie = $webSession.Cookies.GetCookies($baseUri) |
312         Where-Object { $_.Name -eq "AuthCookie" } |
313         Select-Object -First 1
314     $token = $cookie.Value
315 }
316 if (-not $token) {
317     throw "Login to $Server succeeded but no token was returned."
318 }
319
320 [pscustomobject]@{
321     Server          = $Server
322     BaseUri         = $baseUri
323     Username        = $Username
324     Token           = $token
325     IssuedAt        = Get-Date
326     TokenRefreshMinutes = $TokenRefreshMinutes
327     TrustAllCertificates = [bool]$TrustAllCertificates
328 }
329 }
330
331 function Update-NexusDashboardToken {
332     # ND tokens are short lived (20 minutes by default), so a long run - such as
333     # walking a large switch inventory - has to renew the token before it lapses.
334     # The refreshed token replaces the old one on the session object in place.
335     param(
336         [Parameter(Mandatory = $true)] [pscustomobject]$Session
337     )
338
339     $params = @{
340         Uri      = "$($Session.BaseUri)/refresh"
341         Method    = "Post"
342         Headers   = @{ "Authorization" = "Bearer $($Session.Token)" }
343         ContentType = "application/json"
344     }
345     if ($Session.TrustAllCertificates) {
346         $params["SkipCertificateCheck"] = $true
347     }
348
349     try {
350         $response = Invoke-RestMethod @params
351     }
352     catch {
353         throw "Token refresh on $($Session.Server) failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
354     }
355
356     $token = $response.jwttoken
357     if (-not $token) { $token = $response.token }
358     if (-not $token) {
359         throw "Token refresh on $($Session.Server) returned no token."
360     }
361
362     $Session.Token = $token
363     $Session.IssuedAt = Get-Date
364 }
365
366 function Invoke-NexusDashboardApi {
367     param(
368         [Parameter(Mandatory = $true)] [pscustomobject]$Session,
369         [Parameter(Mandatory = $true)] [string]$Path,
370         [ValidateSet("Get", "Post", "Put", "Delete")] [string]$Method = "Get",
371         $Body
372     )
373
374     if (((Get-Date) - $Session.IssuedAt).TotalMinutes -ge $Session.TokenRefreshMinutes) {
375         Update-NexusDashboardToken -Session $Session

```

```

376     Write-Log "Token refreshed on $($Session.Server)."
377 }
378
379 $params = @{
380     Uri      = "$($Session.BaseUri)/$($Path.TrimStart('/'))"
381     Method   = $Method
382     Headers = @{ "Authorization" = "Bearer $($Session.Token)" }
383 }
384 if ($null -ne $Body) {
385     $params["Body"] = if ($Body -is [string]) { $Body } else { $Body | ConvertTo-Json -Depth 10 }
386     $params["ContentType"] = "application/json"
387 }
388 if ($Session.TrustAllCertificates) {
389     $params["SkipCertificateCheck"] = $true
390 }
391
392 try {
393     Invoke-RestMethod @params
394 }
395 catch {
396     throw "$Method $Path on $($Session.Server) failed: $(Get-NexusDashboardErrorMessage -ErrorRecord $_)"
397 }
398 }
399
400 function Disconnect-NexusDashboard {
401     param(
402         [Parameter(Mandatory = $true)] [pscustomobject]$Session
403     )
404
405     # Logging out invalidates the token right away instead of leaving it valid
406     # until it expires. A failure here is not worth failing the run over.
407     try {
408         Invoke-NexusDashboardApi -Session $Session -Path "/logout" -Method Post | Out-Null
409         Write-Log "Logged out of $($Session.Server)."
410     }
411     catch {
412         Write-Log "Logout from $($Session.Server) failed: $(($_.Exception.Message))" -Level WARN
413     }
414 }
415
416 # --- Function: Read the switch inventory from Nexus Dashboard ---
417 function Get-NexusDashboardSwitches {
418     <#
419     .SYNOPSIS
420         Get the switches managed by the Fabric Controller (NDFC) service
421     .PARAMETER Session
422         The session returned by Connect-NexusDashboard
423     .OUTPUTS
424         One object per switch: Name, IPAddress, Model, Serial, Fabric, Role,
425         Version and Status
426     #>
427     param(
428         [Parameter(Mandatory = $true)] [pscustomobject]$Session
429     )
430
431     # The response is a JSON array; @() unrolls it whether it arrives as one
432     # array object or one switch at a time.
433     $rawSwitches = Invoke-NexusDashboardApi -Session $Session `
434         -Path "/appcenter/cisco/ndfc/api/v1/lan-fabric/rest/inventory/allswitches"
435
436     @($rawSwitches) | Where-Object { $_ } | ForEach-Object {
437         [pscustomobject]@{
438             Name      = $_.logicalName
439             IPAddress = $_.ipAddress
440             Model     = $_.model

```

```

441         Serial    = $_.serialNumber
442         Fabric    = $_.fabricName
443         Role      = $_.switchRole
444         Version   = $_.release
445         Status    = $_.status
446     }
447 }
448 }
449
450 # --- Function: End the Nexus Dashboard and SWIS sessions ---
451 function Stop-Sessions {
452     <#
453     .SYNOPSIS
454         End the Nexus Dashboard and SWIS sessions opened by this script
455     .DESCRIPTION
456         Nexus Dashboard publishes a logout endpoint, so the token is revoked
457         rather than left to expire.
458
459         SWIS has no Disconnect-Swis cmdlet, but Connect-Swis returns an
460         InfoServiceProxy, which is IDisposable, so the connection can be
461         closed explicitly instead of being left to the finalizer.
462
463         Safe to call more than once - each session is ended only once.
464     #>
465     if ($script:ndSession) {
466         Disconnect-NexusDashboard -Session $script:ndSession
467         $script:ndSession = $null
468     }
469
470     if ($script:swis) {
471         try {
472             $script:swis.Close()
473             Write-Log "Closed the SWIS connection."
474         } catch {
475             Write-Log "Failed to close the SWIS connection: $($_.Exception.Message)" -Level WARN
476         }
477         $script:swis = $null
478     }
479 }
480
481 # --- Counters for the summary at the end of the run ---
482 $script:stats = [ordered]@{
483     ComponentsAdded    = 0
484     ComponentsExisting = 0
485     ComponentsSkipped  = 0
486     ComponentsFailed   = 0
487     InterfacesAdded    = 0
488     PollersAdded       = 0
489 }
490
491 Write-Log "==== Run started: Nexus Dashboard $NXServer -> SolarWinds $SwisHost ====="
492
493 # --- Connect to Nexus Dashboard ---
494 # A saved credential for unattended runs, otherwise a prompt that keeps the
495 # password as a SecureString.
496 try {
497     if ($NXCredentialPath) {
498         $ndCredential = Import-Clixml -Path $NXCredentialPath -ErrorAction Stop
499     }
500     else {
501         $ndCredential = Get-Credential -Message "Nexus Dashboard API credentials"
502     }
503     if (-not $ndCredential) {
504         throw "No credential was given."
505     }

```

```

506
507     $ndSession = Connect-NexusDashboard -Server $NXServer `
508         -Username $ndCredential.UserName `
509         -Password $ndCredential.Password `
510         -Domain $NXDomain `
511         -TokenRefreshMinutes $TokenRefreshMinutes `
512         -TrustAllCertificates:$TrustAllCertificates
513     Write-Log "Connected to Nexus Dashboard $NXServer as $($ndSession.Username) (domain $NXDomain)."
514 } catch {
515     Write-Log "Failed to authenticate with Nexus Dashboard: $($_.Exception.Message)" -Level ERROR
516     Stop-Sessions
517     exit 1
518 }
519
520 # Call function "Get-NexusDashboardSwitches"
521 try {
522     $switches = @(Get-NexusDashboardSwitches -Session $ndSession)
523 } catch {
524     Write-Log "Failed to retrieve switches from Nexus Dashboard: $($_.Exception.Message)" -Level ERROR
525     Stop-Sessions
526     exit 1
527 }
528
529 # The switch list is all that is needed from Nexus Dashboard, so log out now
530 # instead of keeping the token alive while the nodes are added.
531 Stop-Sessions
532
533 $withIp = @($switches | Where-Object { $_.IPAddress }).Count
534 Write-Log "Nexus Dashboard returned $($switches.Count) switch[es], $withIp with a management IP address."
535 foreach ($device in $switches) {
536     Write-Log "    switch          | name=$(($device.Name) | IP=$(($device.IPAddress) | model=$(($device.Model) |
537         serial=$(($device.Serial) | fabric=$(($device.Fabric) | status=$(($device.Status)" -NoConsole
538 }
539
540 if ($switches.Count -eq 0) {
541     Write-Log "Nexus Dashboard returned no switches - nothing to process." -Level WARN
542     exit
543 }
544
545 # --- Connect to SWIS ---
546 try {
547     $swisCredential = Import-Clixml -Path $SwisCredentialPath -ErrorAction Stop
548     $swis = Connect-Swis -Hostname $SwisHost -Credential $swisCredential
549     Write-Log "Connected to SolarWinds Information Service (SWIS) on $SwisHost."
550 } catch {
551     Write-Log "Failed to connect to SWIS: $($_.Exception.Message)" -Level ERROR
552     Stop-Sessions
553     exit 1
554 }
555
556 # --- Build the list of components to process ---
557 # Build $components from the switches returned by Nexus Dashboard: the
558 # management IP address is what the node is added by, and the switch name is
559 # kept as the node caption. A switch without a valid IP address is skipped
560 # with a warning.
561 $components = foreach ($device in $switches) {
562     $address = $null
563     $ip = "$($device.IPAddress)".Trim()
564
565     if (-not $ip -or -not [System.Net.IPAddress]::TryParse($ip, [ref]$address)) {
566         Write-Log "Skipping '$($device.Name)' - no usable management IP address ('$($device.IPAddress)')." -
567             Level WARN
568         Write-Log "    component          | name=$(($device.Name) | IP=$(($device.IPAddress) | status=skipped" -
569             NoConsole
570         $script:stats.ComponentsSkipped++
571     }
572 }

```

```

568     continue
569 }
570
571 @{
572     IPAddress    = $address.ToString()
573     Caption      = $device.Name
574     EngineID     = $EngineID
575     SNMPVersion  = $SNMPVersion
576     DNS          = ""
577     SysName      = ""
578     Community    = $SNMPCommunity
579 }
580 }
581
582 # --- Function: Add a Node [Component] ---
583 function Add-Component {
584     param($component)
585
586     $newNodeProps = @{
587         IPAddress    = $component.IPAddress
588         EngineID     = $component.EngineID
589         Caption      = $component.Caption
590         # SNMP v2 specific
591         ObjectSubType = "SNMP"
592         SNMPVersion  = $component.SNMPVersion
593         DNS          = $component.DNS
594         SysName      = $component.SysName
595         Community    = $component.Community
596         # === default values ===
597         # EntityType   = 'Orion.Nodes'
598         # DynamicIP    = $false
599         # PollInterval = 120
600         # RediscoveryInterval = 30
601         # StatCollection = 10
602     }
603
604     $newNodeUri = New-SwisObject -SwisConnection $swis -EntityType "Orion.Nodes" -Properties $newNodeProps
605     $nodeProps  = Get-SwisObject -SwisConnection $swis -Uri $newNodeUri
606
607     # Register the standard set of pollers for the node
608     $poller = @{
609         NetObject      = "N:" + $nodeProps["NodeID"]
610         NetObjectType   = "N"
611         NetObjectID     = $nodeProps["NodeID"]
612     }
613
614     foreach ($pollerType in @(
615         "N.Status.ICMP.Native",
616         "N.ResponseTime.ICMP.Native",
617         "N.Details.SNMP.Generic",
618         "N.Uptime.SNMP.Generic"
619     )) {
620         $poller["PollerType"] = $pollerType
621         New-SwisObject -SwisConnection $swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
622         Write-Log " poller added      | NodeID=$(($nodeProps['NodeID'])) | pollerType=$pollerType" -NoConsole
623         $script:stats.PollersAdded++
624     }
625
626     return $nodeProps["NodeID"]
627 }
628
629 # --- Function: Discover and Add Interfaces on a Node ---
630 function Add-DiscoveredInterfaces {
631     param($nodeId)
632

```

```

633 # Discover interfaces on the node
634 $discovered = Invoke-SwisVerb $swis Orion.NPM.Interfaces DiscoverInterfacesOnNode $nodeId
635
636 if ($discovered.Result -ne "Succeed") {
637     Write-Log " Interface discovery failed for node $nodeId." -Level ERROR
638     return
639 }
640
641 # Keep only the interfaces that match -InterfaceFilter and are
642 # operationally up (ifOperStatus 1); everything else is removed before
643 # the add.
644 #
645 # The node list is materialised with @() first: RemoveChild shrinks the
646 # live XmlNodeList, and removing from it while the pipeline is still
647 # enumerating it skips nodes.
648 @($discovered.DiscoveredInterfaces.DiscoveredLiteInterface) | Where-Object {
649     $_.Caption.InnerText -notmatch $InterfaceFilter -or
650     $_.ifOperStatus -ne '1'
651 } | ForEach-Object { $discovered.DiscoveredInterfaces.RemoveChild($_) | Out-Null }
652
653 # Where-Object drops the null that an emptied node list yields: @($null)
654 # has a Count of 1, so without it a run that filtered every interface away
655 # would report one interface and still call the add verb with nothing.
656 $interfaceCount = @($discovered.DiscoveredInterfaces.DiscoveredLiteInterface | Where-Object { $_ }).Count
657
658 if ($interfaceCount -eq 0) {
659     Write-Log " No interfaces left to add for node $($nodeId) after filtering."
660     return
661 }
662
663 # Add the remaining interfaces. The node's interfaces are read before and
664 # after the add, so the log names exactly the interfaces that were added
665 # and not the ones that were already monitored.
666 $interfaceQuery = "SELECT InterfaceID, Caption FROM Orion.NPM.Interfaces WHERE NodeID = @nodeId"
667 try {
668     $before = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery -Parameters @{ nodeID =
669 $nodeId } | ForEach-Object { $_.InterfaceID })
670
671     Invoke-SwisVerb $swis Orion.NPM.Interfaces AddInterfacesOnNode @($nodeId,
672 $discovered.DiscoveredInterfaces, "AddDefaultPollers") | Out-Null
673
674     $added = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery -Parameters @{ nodeID =
675 $nodeId } |
676     Where-Object { $before -notcontains $_.InterfaceID })
677
678     if ($added.Count -eq 0) {
679         Write-Log " No new interfaces for node $($nodeId) - the $interfaceCount matching interface[s]
680 were already monitored."
681         return
682     }
683
684     Write-Log " Added $($added.Count) interface[s] for node $($nodeId)."
685     foreach ($interface in $added) {
686         Write-Log " interface added | NodeID=$nodeId | InterfaceID=$(($interface.InterfaceID) | caption=$(
687 $interface.Caption)" -NoConsole
688     }
689     $script:stats.InterfacesAdded += $added.Count
690 } catch {
691     Write-Log " Failed to add interfaces for node $($nodeId): $($_.Exception.Message)" -Level ERROR
692 }
693 }
694
695 # --- Main Loop: Process each component ---
696 foreach ($component in $components) {
697     Write-Log "Processing component $($component.Caption) ($($component.IPAddress))..."

```

```

693
694     try {
695         # Check if node already exists
696         $existing = Get-SwisData -SwisConnection $swis -Query "SELECT NodeID FROM Orion.Nodes WHERE IPAddress
= @ip" -Parameters @{ ip = $component.IPAddress }
697
698         if ($existing) {
699             $nodeId = @($existing)[0]
700             Write-Log " Node already exists [NodeID $nodeId], skipping add."
701             Write-Log "   component          | name=$(($component.Caption) | IP=$(($component.IPAddress) |
status=exists | NodeID=$nodeId" -NoConsole
702                 $script:stats.ComponentsExisting++
703         }
704         else {
705             $nodeId = Add-Component $component
706             Write-Log " Added node [NodeID $nodeId]."
707             Write-Log "   component          | name=$(($component.Caption) | IP=$(($component.IPAddress) |
status=added | NodeID=$nodeId" -NoConsole
708                 $script:stats.ComponentsAdded++
709         }
710
711         # Discover and add interfaces for the node (whether new or existing)
712         Add-DiscoveredInterfaces $nodeId
713
714     } catch {
715         Write-Log " Failed to process $(($component.Caption) ($($component.IPAddress))): $
($_.Exception.Message)" -Level ERROR
716         Write-Log "   component          | name=$(($component.Caption) | IP=$(($component.IPAddress) |
status=failed" -NoConsole
717                 $script:stats.ComponentsFailed++
718     }
719 }
720
721 # --- End the SWIS session ---
722 Stop-Sessions
723
724 $summary = "Summary: $($script:stats.ComponentsAdded) component[s] added, " +
725     "$($script:stats.ComponentsExisting) already existed, " +
726     "$($script:stats.ComponentsSkipped) skipped, " +
727     "$($script:stats.ComponentsFailed) failed; " +
728     "$($script:stats.InterfacesAdded) interface[s] and " +
729     "$($script:stats.PollersAdded) poller[s] added."
730 Write-Log $summary
731
732 Write-Log "Script completed."
733 Write-Log "=== Run finished ===" -NoConsole
734
735 # A scheduled task sees the run as failed when any component failed.
736 if ($script:stats.ComponentsFailed -gt 0) { exit 1 }

```